Federated TinyML on ESP32: A Resource-Constrained Framework for IoT Health Monitoring

1st Chandan B Ram
Department of Computer Science and Engineering
Siddaganga Institute of Technology
Tumakuru, India
chandubram2001@gmail.com

2nd Nirmala. M B
Department of Computer Science and Engineering
Siddaganga Institute of Technology
Tumakuru, India mbnirmala@sit.ac.in

3rd Swathi Papanna Gari
Department of Computer Science and Engineering
Siddaganga Institute of Technology
Tumakuru, India
swathip@sit.ac.in

2025 2nd International Conference on Software, Systems and Information Technology (SSITCON) | 979-8-3315-2623-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/SSITCON66133.2025.11342233

***Abstract*—The combination of Tiny Machine Learning (TinyML) with Federated Learning (FL) enables intelligent analytics on low-power microcontrollers. While prior often rely on resource-rich devices (e.g., Raspberry Pi, Jetson), very few have demonstrated FL on low-power devices. This research develops a resource-limited federated learning system which operates on ESP32 devices to perform real-time health monitoring. A lightweight neural network processed parameters like blood oxygen saturation (SpO_2), heart rate and temperature. The devices run local anomaly detection while sending model updates to build a global model through federated averaging. Unlike centralized TinyML deployments, our framework maintains privacy by keeping all raw health data on the device. The proposed system reaches an average classification accuracy of 98% while maintaining inference latency under 100 ms and keeping the quantized model size below 3 KB to support deployment on memory-restricted hardware. These results demonstrate that federated aggregation preserves accuracy while reducing centralized storage needs enabling privacy-preserving and resource-efficient solution.**

Keywords—tiny machine learning (tinyML), federated learning (FL), edge computing, esp32 microcontroller, IoT health monitoring, resource-constrained devices.

I. Introduction

The quick expansion of Internet of Things (IoT) medical equipment has created a need for analytics systems that operate directly on edge hardware with low-latency and privacy protection capabilities. The implementation of Tiny Machine Learning (TinyML) makes real-time intelligence possible through its ability to execute compact neural models on minimal resource microcontrollers. Heydari and Mahmoud (2025) explain that TinyML provides essential benefits for cloud inference over centralized model training because it serves domains needing immediate responses and privacy protection despite limited memory and compute resources [1].

The traditional TinyML deployment model faces scalability issues because it depends on centralized model training which also jeopardizes data confidentiality. The FL framework solves this problem by training models across multiple devices while protecting original data from exchange. Embedded systems benefit from FL but most existing implementations focus on high-end devices such as Raspberry Pi or Jetson platforms [2][3]. Although several recent studies have attempted FL integration with constrained MCUs, very few demonstrate a fully operational system on ESP32-class devices for healthcare, leaving a clear gap in real-world validation.

Research initiatives now work to merge TinyML with FL in microcontroller platforms. A distributed training process with asynchronous gossip-based method helps improve convergence when devices have decentralized network connections [4]. A recent breakthrough in multi-class TinyML classifiers demonstrates that constrained devices can perform efficient learning under limited memory conditions [5]. The literature currently lacks proof of real-time health monitoring using federated TinyML systems that function on common microcontroller platforms such as ESP32.

Our research addresses this gap by establishing a TinyML + FL system on ESP32 devices to detect anomalies from physiological signals (SpO_2, heart rate, temperature) at the edge. The system combines a small neural model (TFLite <3 KB) with federated averaging distributed across multiple ESP32 clients to achieve real-time inference under 100 ms. Our decentralized training method maintains approximately 98% accuracy across a 1,000-sample dataset while operating within decentralized parameters. Given the dataset size, this work should be interpreted as an early proof-of-concept rather than a fully validated medical system.

A. Contributions of this paper:

- A practical TinyML model architecture deployed on ESP32 microcontrollers for real-time anomaly detection in health monitoring.
- The research integrates ESP32 devices into FedAvg-based federated learning to facilitate collaborative training while maintaining minimal performance reduction.
- A comprehensive evaluation reporting not only accuracy but also latency, model footprint, and prototype hardware performance, addressing both algorithmic and system-level challenges.

B. Organization of the Paper

The following sections make up the rest of this paper. The paper reviews TinyML and federated learning research in embedded and IoT environments in Section II. The proposed framework is explained in Section III through descriptions of the hardware setup and dataset and lightweight neural model architecture that works on ESP32. The federated learning workflow and experimental evaluation results show accuracy and latency and model footprint in Section IV. The paper examines the practicality and boundaries of implementing TinyML and FL on devices with limited resources in Section V. The paper concludes in Section VI with future research directions.

II. Literature Review

Recent researches have focused on uniting distributed intelligence with extreme resource-limited environments through advances in FL technology and TinyML approaches. The transition to ultra-low-power IoT devices and

979-8-3315-2623-8/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:10:53 UTC from IEEE Xplore. Restrictions apply.

microcontrollers requires new specialized methods for training and deployment alongside inference operations because traditional FL frameworks operate under moderate computing and communication capacity assumptions. Existing research investigates FL protocols designed for lightweight operations while optimizing hardware and developing compiler-level innovations to demonstrate industrial applications and healthcare and vision domain use cases. Research reviews show technical capabilities with performance standards that help new approaches establish their positions by comparing accuracy levels to latency, memory usage and scalability in operational conditions.

Several studies highlight lightweight federated protocols [6], edge inference feasibility on ESP32 [7], surveys of lightweight FL [8], LoRa-based FL implementations [9], and neural network library integration for FL prototyping [10]. However, most of these remain at a conceptual or prototype stage without demonstrating a full-stack system for real-time health monitoring on low-cost ESP32 devices.

A framework to train and communicate with IoT devices through lightweight methods which minimized local update memory requirements while maintaining acceptable convergence performance within resource constraints [6]. The execution of object detection on ESP32 microcontrollers demonstrated successful results that reduced latency more than cloud-based inference methods thereby proving microcontroller-level deployments are feasible [7]. Research studies on lightweight federated learning have identified model compression along with sparse updates and device selection as key design elements yet they emphasize theoretical aspects instead of system implementation details [8].

The network layer implementation of LoRa-based federated learning used Portenta-class nodes and LoRa32 routers to perform distributed training on ultra-low-rate wireless links [9]. The integration of portable neural network libraries into embedded federated setups has allowed prototype-level client training that demonstrates better communication performance than centralized alternatives through measured results [10]. The integration of TinyML with transfer learning techniques enabled partial on-device model updates which reached cloud training accuracy at much lower bandwidth costs [11].

The implementation of approximate convolution kernels that consider operand significance resulted in a 20% latency decrease for STM32-class devices while maintaining high accuracy levels [12]. The implementation of transfer learning-based methods for embedded vibration sensing on TinyML devices produced superior domain generalization results during machine fault diagnosis [13]. Real-time motor vibration detection reached over 96% accuracy through the integration of MCU-based feature extraction with TinyML classifiers in industrial monitoring applications [14].

Ultra-lightweight pipelines have also emerged. An ESP32-CAM system using HOG features with Random Forest classifier demonstrated 80 FPS image recognition while using only 1 kB of RAM to show the trade-off between deep learning capabilities and extreme efficiency [15]. The benchmarking process gained support through on-board evaluation toolkits that allow standardized performance testing across various MCU hardware platforms [16]. Compiler-level innovations that use tensor-splitting techniques allow deeper CNNs to run on limited SRAM environments through sequential tensor processing which results in six-fold memory efficiency with only minimal latency costs [17].

TABLE I. SUMMARY OF RECENT ADVANCES IN TINYML AND FEDERATED LEARNING FOR RESOURCE-CONSTRAINED IOT

Layer / Scope	Technical focus	Resource dimension	Strengths	Limitations	Ref.
FL protocol	Lightweight client update mechanisms	Communication, memory	Enables MCU-class participation with reduced load	Slower convergence, device heterogeneity	[6]
Edge deployment	ESP32-based on-device inference	Latency, hardware cost	Real-time gains on low-cost hardware	Limited model depth, moderate accuracy	[7]
FL landscape	Survey of lightweight FL strategies	Conceptual frameworks	Maps design axes such as compression & sparsity	Lacks empirical evaluation	[8]
FL over networks	Federated learning over LoRa mesh	Communication bandwidth	Demonstrates feasibility under ultra-low data rates	High latency per round, interference sensitivity	[9]
FL library	Neural network library integration	Portability, prototyping	Eases embedded FL deployment with reusable stacks	Not scalable to larger models	[10]
FL + TL	On-board transfer learning with aggregation	Accuracy, bandwidth	Retains accuracy close to centralized training	Risk of client drift, retraining overhead	[11]
Kernel optimization	Approximate convolution kernels	Latency, computation	~20% inference speed-up on MCUs	Potential accuracy degradation	[12]
Edge transfer learning	Domain generalization for fault diagnosis	Accuracy, generalization	Effective for cross-domain machine faults	Memory burden of additional layers	[13]
Edge sensing	Vibration anomaly detection via TinyML	Real-time monitoring	~96% accuracy, robust deployment on MCU	Sensitive to noise and dataset shifts	[14]
Ultra-light pipeline	HOG + Random Forest on ESP32	Memory, speed	80 FPS, only ~1 kB RAM usage	Limited adaptability, manual feature design	[15]
Evaluation framework	Universal TinyML benchmarking toolkit	Standardization	Enables reproducible cross-device performance	May not capture workload diversity	[16]
Compiler/runtime	Tensor-splitting for memory efficiency	Memory optimization	5.9× SRAM efficiency with accuracy preserved	Added runtime complexity, minor latency costs	[17]

In summary, while many advances exist, they either focus on industrial monitoring, vibration sensing, or image recognition, or they use stronger hardware platforms. Our work differentiates itself by targeting federated TinyML anomaly detection in healthcare on ESP32, validated with hardware deployment, making it both novel and practical. A comparative summary of recent TinyML and federated learning approaches for constrained IoT devices is presented in Table I. and Table II.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:10:53 UTC from IEEE Xplore. Restrictions apply.

TABLE II. COMPARATIVE OVERVIEW OF LITERATURE IN TINYML AND FEDERATED LEARNING

Ref.	Primary Contribution	Method Family	Evaluation Focus
[6]	Lightweight client update mechanisms	Federated Learning (FL protocol)	Communication & memory efficiency
[7]	ESP32-based on-device inference	Edge Computing	Latency, hardware cost
[8]	Survey of lightweight FL strategies	Literature Review	Conceptual frameworks (compression, sparsity)
[9]	FL over LoRa mesh network	Federated Learning over networks	Bandwidth and latency trade-offs
[10]	Neural network library integration	FL prototyping frameworks	Portability & scalability
[11]	On-board transfer learning with FL aggregation	Federated + Transfer Learning	Accuracy & client drift handling
[12]	Approximate convolution kernels	Model optimization (kernel-level)	Inference latency & computation
[13]	Domain generalization for fault diagnosis	Transfer Learning + Edge ML	Accuracy & generalization ability
[14]	Vibration anomaly detection	TinyML sensing applications	Real-time monitoring & robustness
[15]	Ultra-light HOG + Random Forest pipeline	Classical ML on TinyML	FPS throughput & memory usage
[16]	Universal benchmarking toolkit	Evaluation framework	Cross-device performance reproducibility
[17]	Tensor-splitting compiler/runtime	Compiler & runtime optimization	Memory efficiency vs. latency

III. POPOSED SYSTEM

The system combines Tiny Machine Learning (TinyML) with Federated Learning (FL) to create a privacy-focused IoT health monitoring framework which runs on ESP32 microcontrollers. The system maintains its operation within harsh resource limitations to deliver precise real-time anomaly detection for physiological signals.

A. *System Architecture*

The system implements a client–server federated learning architecture (Figure 1).

ESP32 Clients (Edge Devices):

- The equipment includes sensors which detect SpO_2 readings together with heart rate measurements and temperature recordings.
- Each device performs on-device preprocessing and TinyML inference.
- Private data processing occurs on each device through local training without sharing raw signal information.

FL Server (Aggregator):

- The FL server accepts model weights through multiple ESP32 clients.
- The server executes federated averaging (FedAvg) to build the global model.
- The server distributes the updated model through the network to all connected clients.

Communication Layer:

- MQTT protocol is used.

The model parameters and gradients make up the only data transmitted between nodes thus reducing bandwidth usage and protecting user privacy.

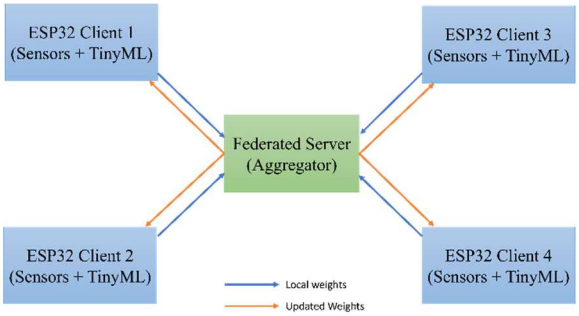


Fig. 1. Federated Learning Architecture

B. *Data Flow*

- The acquisition of signals occurs when sensors record SpO_2, heart rate, and temperature.
- The preprocessing step uses z-score normalization for input data.
- Each ESP32 unit runs an anomaly detection algorithm with quantized TinyML models during local inference.
- Local Training: Devices update model parameters using new samples.
- Federated Aggregation: Server averages updates.
- The updated global model gets redistributed to all devices through the deployment stage.
- The learning process continues without breaching patient data privacy through this recurring cycle.

C. *Model Architecture*

The TinyML model operates on ESP32 devices with limited memory capacity (less than 3 KB) while achieving high accuracy levels of approximately 98%. Although the architecture (3–12–6–1) is intentionally shallow, this simplicity reflects a deliberate trade-off between accuracy and resource-efficiency for MCU deployment.

Layer-by-layer architecture (Figure 2):

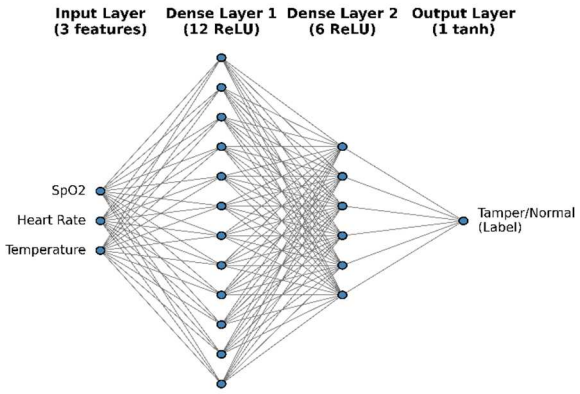


Fig. 2. Neural Network Model Architecture.

- The model accepts normalized sensor data through its three-input neuron layer .
- [SpO_2,HeartRate,Temperature].
- Dense Layer 1 (12 neurons, ReLU): Extracts nonlinear feature interactions.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:10:53 UTC from IEEE Xplore. Restrictions apply.

- The network uses Dense Layer 2 (6 neurons, ReLU) to generate compact intermediate feature representations.
- The output layer containing one neuron with tanh activation generates an anomaly score.

D. System Features

- Resource Efficiency: Optimized using TFLite quantization for deployment [18] [19].
- The FL cycle accepts additional ESP32 nodes without disrupting its operation.
- The system protects privacy through data localization since it keeps raw sensor information on devices and distributes model updates only [20].
- Low Latency: Average inference <100 ms, suitable for real-time monitoring.

E. Methodology

The research methodology covers the complete process of developing and optimizing and deploying a lightweight federated TinyML model for health anomaly detection on ESP32 microcontrollers. The research process includes dataset preparation and feature preprocessing and neural model design and training and evaluation and quantization and microcontroller firmware deployment.

1) Dataset Preparation and Preprocessing: The experimental dataset consisted of 1,000 physiological sensor readings of SpO_2, heart rate and temperature which received binary anomaly detection labels.

The raw input features were represented as: $X = \{x1, x2, x3\} = \{SpO^2, HeartRate, Temperature\}$ ans $y \in \{0,1\}$

Since the feature values varied in scale, standard normalization was applied as in Equation (1)

$$X' = \frac{X - \mu}{\sigma} \tag{1}$$

Where σ is the standard deviation. The standardization process made all features equally important for training and accelerated the convergence process.

The dataset split into 80% training data and 20% testing data to maintain evaluation accuracy. No explicit noise handling or missing-value imputation was required due to dataset constraints, but such techniques would be necessary for larger, real-world deployments.

2) Neural Network Model Architecture: A lightweight feedforward neural network was designed for compatibility with ESP32 resource limits. The architecture is illustrated in Figure 2 and can be mathematically represented as follows and depicted from Equations (2) – (5):

Input Layer:

$$h^{(0)} = X' \tag{2}$$

Hidden layer 1:

$$h^{(1)} = max\left(0,\ h^{(0)}W^{(1)} + b^{(1)}\right) \tag{3}$$

Hidden layer 2:

$$h^{(2)} = max\left(0,\ h^{(1)}W^{(2)} + b^{(2)}\right) \tag{4}$$

Output Layer (1 neuron, tanh):

$$\hat{y} = tanh\left(h^{(2)}W^{(3)} + b^{(3)}\right) \tag{5}$$

Loss function was defined as Mean Squared Error (MSE) as shown in Equation (6):

$$L = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y_i})^2 \tag{6}$$

The model used the Adam optimizer with a learning rate adjustment for achieving rapid convergence under hardware restrictions.

3) Training and Evaluation: The training process lasted for 30 epochs with 16 samples per batch while using 10% of the training data for validation. The model reached 98% test dataset classification accuracy while maintaining an appropriate inference speed for real-time health monitoring. The model reached convergence through loss and accuracy curve analysis across epochs to prevent overfitting.

4) Model Quantization for TinyML: The trained model underwent TensorFlow Lite post-training quantization to prepare it for deployment on ESP32-class hardware. The quantization process transformed the floating-point model into a 3 KB quantized format which maintained 99% of the original inference accuracy. Quantization reduces weights and activations into 8-bit integer precision, formalized as in Equation (7):

$$w_q = round\left(\frac{w_f}{s}\right) + Z \tag{7}$$

Where w_q is the quantized weight, s is the scaling factor, and z is the zero-point offset. This makes inference feasible under ESP32's limited SRAM and flash constraints. The corresponding dequantization step during inference reconstructs approximate floating-point values as shown in Equation (8):

$$w_f \approx S \cdot \left(w_q - Z\right) \tag{8}$$

5) Conversion for Microcontroller Deployment: The quantized model received conversion to a C-style header file (.h) through binary-to-header transformation after being saved as.tflite format. The model integration into firmware as a static array occurred during this step as const unsigned char model [] = {0x20,0x00,0x4F,…}

The model storage in flash memory (PROGMEM) minimized RAM requirements which allowed deployment on ESP32 devices without external storage needs.

The process completed the data-to-trained model-to-quantized TFlite-to-C header.h-to-ESP32 firmware pipeline for achieving end-to-end portability and efficiency.

IV. Results and Discussion

The TinyML health monitoring system based on federated architecture was implemented and validated on ESP32 microcontroller platform. The hardware prototype in Figure 3 integrates an ESP32 board with peripheral modules which

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:10:53 UTC from IEEE Xplore. Restrictions apply.

includes DHT sensor for temperature and humidity measurement, SpO_2 and heart rate sensor, status LEDs, and a piezo buzzer for anomaly alerts. A compact and low-cost edge computing node for real-time health monitoring that is energy-efficient is demonstrated in the circuit layout.

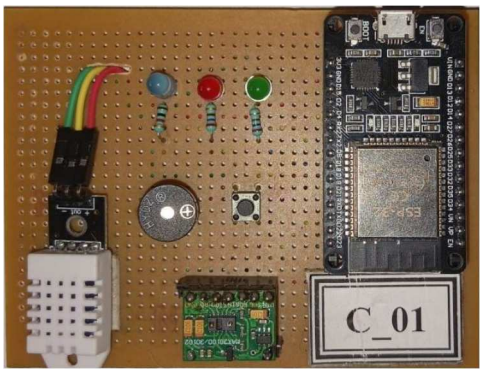


Fig. 3. Prototype hardware setup of the ESP32-based TinyML health monitoring system

On the software side, the designed lightweight neural network was trained using 1,000 samples of physiological data (SpO_2, heart rate, and temperature). The dataset was normalized, split into training and testing sets, and trained for 30 epochs. Comparison of the work with other TinyML-FL is given in Table III. The training and validation performance curves are shown in Figures 4 and 5. Figure 4 shows Loss curve over epochs and Figure 5 shows Accuracy curve over epochs.

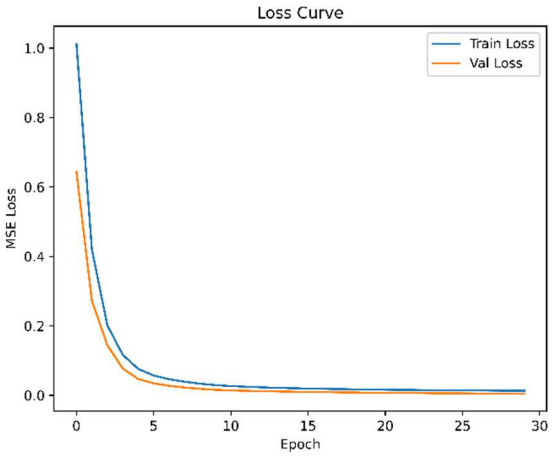


Fig. 4. Training and Validation Loss Curve

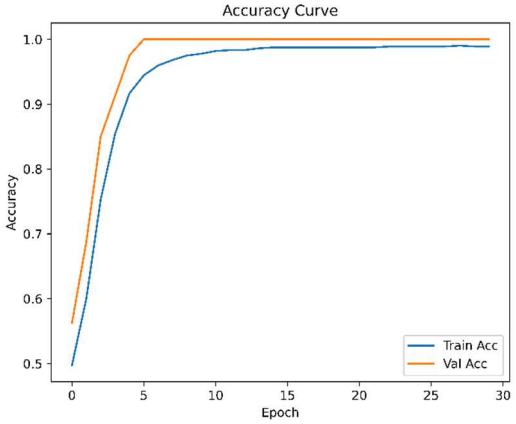


Fig. 5. Training and Validation Accuracy Curve

TABLE III. COMPARISION OF TINYML -FL SYSTEMS

Paper	Device	Dataset	Key Results
[7]	ESP32-CAM	ImageNet	Acc: 0.76 (edge), 0.98 (cloud); Loop: 641–6456 ms
[9]	Portenta + TTGO-LORA32	480 voice samples	Acc: A:100%, B:62.5%, C:50%; TX: 25–124 min; Energy: 52–165 mA
This work	ESP32	1,000 readings	Acc: 98%; Latency: <100 ms; Model size: <3 KB

The results highlight the following observations:

- The training and validation loss decreased rapidly within the first 10 epochs before reaching near zero after 20 epochs. The fast convergence of the lightweight architecture on small datasets is supported by the result.
- High Accuracy: The system achieved a final classification accuracy of approximately 98%, with validation accuracy closely tracking training accuracy, indicating strong generalization performance.
- The quantized TensorFlow Lite model had a size of less than 3 KB which made it compatible with ESP32's limited flash and SRAM. The inference latency was consistently under 100 ms, validating the feasibility of real-time anomaly detection at the edge.
- The hardware testing confirmed that anomaly events (e.g., abnormal heart rate or temperature spikes) were detected locally, with corresponding LED/buzzer alerts triggered, while model updates were aggregated using federated learning. Scaling experiments across many heterogeneous devices and communication conditions remain to be investigated.

V. CONCLUSION

The research developed a federated TinyML system which detects health anomalies on ESP32 microcontrollers through 98% accurate results with less than 100 ms inference delay and model size under 3 KB. The system uses TinyML together with FL to provide real-time health signal monitoring that preserves privacy while meeting strict resource requirements. The federated system maintains raw data on devices for confidentiality protection while global aggregation techniques preserve model performance. The experimental results demonstrated the feasibility of using low-cost edge nodes with energy efficiency as an initial step toward scalable healthcare monitoring, though large-scale deployment and validation remain open challenges. Future research will focus on expanding dataset sizes and developing secure aggregation methods and communication optimization techniques to enhance federated TinyML applications in IoT health monitoring.

REFERENCES

[1] S. Heydari and Q. H. Mahmoud, "Tiny machine learning and on-device inference: A survey of applications, challenges, and future directions," Sensors, vol. 25, no. 10, p. 3191, 2025, doi: 10.3390/s25103191.

[2] K. Radhakrishnan, D. Ramakrishnan, and R. A. Freeda, "Federated learning for artificial intelligence in embedded systems," ICCK Trans. Emerg. Top. Artif. Intell., vol. 2, no. 2, pp. 91–115, 2025, doi: 10.62762/TETAI.2025.440076.

[3] C. N. da Silva and C. V. S. Prazeres, "Tiny federated learning for constrained sensors: A systematic literature review," IEEE Sensors Reviews, vol. 2, pp. 17–31, 2025, doi: 10.1109/SR.2025.3548547.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:10:53 UTC from IEEE Xplore. Restrictions apply.

[4] J. Zhou, J. Tu, and D. Ren, “An asynchronous distributed training algorithm based on gossip,” in Proc. 7th Int. Conf. Big Data Analytics (ICBDA), Guangzhou, China, 2022, pp. 214–217, doi: 10.1109/ICBDA55095.2022.9760359.

[5] T. Puślecki and K. Walkowiak, “A novel one-versus-all approach for multiclass classification in TinyML systems,” IEEE Embedded Systems Letters, vol. 17, no. 2, pp. 71–74, Apr. 2025, doi: 10.1109/LES.2024.3482002.

[6] C. J. N. da Silva, M. L. M. Peixoto, G. B. Figueiredo, and C. V. S. Prazeres, “TinyFed: Lightweight federated learning for constrained IoT devices,” Internet Technology Letters, vol. 8, no. 4, p. e70061, 2025, doi: 10.1002/itl2.70061.

[7] Y.-H. Chang, F.-C. Wu, and H.-W. Lin, “Design and implementation of ESP32-based edge computing for object detection,” Sensors, vol. 25, no. 6, p. 1656, 2025, doi: 10.3390/s25061656.

[8] P. Qi, D. Chiaro, and F. Piccialli, “Small models, big impact: A review on the power of lightweight federated learning,” Future Generation Computer Systems, vol. 162, p. 107484, 2025, doi: 10.1016/j.future.2024.107484.

[9] N. Llisterri Giménez, J. M. Solé, and F. Freitag, “Embedded federated learning over a LoRa mesh network,” Pervasive and Mobile Computing, vol. 93, p. 101819, 2023, doi: 10.1016/j.pmcj.2023.101819.

[10] S. Aatab and F. Freitag, “Integrating a neural network library in embedded federated learning,” in Proc. IEEE 6th Int. Conf. Cloud Comput. Artif. Intell.: Technol. Appl. (CloudTech), Marrakech, Morocco, 2023, pp. 1–7, doi: 10.1109/CloudTech58737.2023.10366148.

[11] M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, and S. Russo, “Federated learning for IoT devices: Enhancing TinyML with on-board training,” Information Fusion, vol. 104, p. 102189, 2024, doi: 10.1016/j.inffus.2023.102189.

[12] G. Armeniakos, G. Mentzos, and D. Soudris, “Accelerating TinyML inference on microcontrollers through approximate kernels,” in Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW), San Francisco, CA, USA, 2024, pp. 177–177, doi: 10.1109/IPDPSW63119.2024.00044.

[13] S. Asutkar, C. Chalke, K. Shivgan, and S. Tallur, “TinyML-enabled edge implementation of transfer learning framework for domain generalization in machine fault diagnosis,” Expert Systems with Applications, vol. 213, no. B, p. 119016, 2023, doi: 10.1016/j.eswa.2022.119016.

[14] S. Arciniegas, D. Rivero, J. Piñan, E. Diaz, and F. Rivas., “IoT device for detecting abnormal vibrations in motors using TinyML,” Discover Internet of Things, vol. 5, p. 41, 2025, doi: 10.1007/s43926-025-00142-4.

[15] B. Sudharsan, S. Salerno, and R. Ranjan, “TinyML-CAM: 80 FPS image recognition in 1 kB RAM,” in Proc. 28th Annu. Int. Conf. Mobile Comput. Netw. (MobiCom ’22), Sydney, NSW, Australia, 2022, pp. 862–864, doi: 10.1145/3495243.3558264.

[16] Z. Huang, K. Zandberg, K. Schleiser, and E. Baccelli, “U-TOE: Universal TinyML On-Board Evaluation Toolkit for Low-Power IoT,” in Proc. 2023 12th IFIP/IEEE Int. Conf. Performance Evaluation and Modeling in Wired and Wireless Networks (PEMWN), Berlin, Germany, 2023, pp. 1–6, doi: 10.23919/PEMWN58813.2023.10304946.

[17] Y.-Y. Liu, H.-S. Zheng, Y. Fang Hu, C.-F. Hsu, and T.-T. Yeh, “TinyTS: Memory-Efficient TinyML Model Compiler Framework on Microcontrollers,” in Proc. 2024 IEEE Int. Symp. High-Performance Computer Architecture (HPCA), Edinburgh, U.K., 2024, pp. 848–860, doi: 10.1109/HPCA57654.2024.00070.

[18] S. S. Saha, S. S. Sandha, and M. Srivastava, “Machine learning for microcontroller-class hardware: A review,” IEEE Sensors Journal, vol. 22, no. 22, pp. 21362–21390, Nov. 2022, doi: 10.1109/JSEN.2022.3210773.

[19] R. Immonen and T. Hämäläinen, “Tiny machine learning for resource-constrained microcontrollers,” Journal of Sensors, Article ID 7437023, pp. 1–11, 2022, doi: 10.1155/2022/7437023.

[20] M. N. A. Ramadan, M. A. H. Ali, S. Y. Khoo, and M. Alkhedher, “Federated learning and TinyML on IoT edge devices: Challenges, advances, and future directions,” ICT Express, vol. 11, no. 4, pp. 754–768, 2025, doi: 10.1016/j.icte.2025.06.008.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 23,2026 at 15:10:53 UTC from IEEE Xplore. Restrictions apply.